AIMERZ.ai
Aim.Act.Achieve

**NOTES**

# Introduction to Object-Oriented Programming

# Introduction to Object-Oriented Programming

## Object-Oriented Programming (OOP)

**Object-Oriented Programming (OOP)** is a programming paradigm that revolves around the concept of "objects." These objects are instances of classes, and they can hold both **data** (attributes) and **methods** (functions or procedures) that operate on the data. OOP is designed to mimic the way we interact with the real world, where everything is represented as an entity or object with its own properties and behaviours.

- **What**: OOP is about structuring programs into classes and objects. A **class** defines a blueprint or template for objects, while an **object** is a specific instance of a class.
- **Why**: OOP promotes organised, modular, and reusable code, making it easier to manage larger and more complex software systems. It also supports code reusability and flexibility.
- **How**: In OOP, you define classes that represent real-world entities or concepts. Each class has attributes (data) and behaviours (methods). You can then create multiple objects from these classes, each representing a specific instance of the entity.

### Example:

A simple example is a Dog class that has attributes like breed, colour, and behaviours like bark(). You can create several objects like dog1 =

Dog("Golden Retriever", "Golden") and dog2 = Dog("Bulldog", "White"), each having its own specific attributes but sharing the common behaviour of barking.

```python
class Dog:
    def __init__(self, breed, color):
        self.breed = breed
        self.color = color

    def bark(self):
        print(f"{self.breed} is barking!")

dog1 = Dog("Golden Retriever", "Golden")
dog2 = Dog("Bulldog", "White")

dog1.bark()  # Output: Golden Retriever is barking!
dog2.bark()  # Output: Bulldog is barking!
```

## Understanding OOP: A Real-World Analogy

Imagine you're organising a library. In a disorganised library (procedural programming), you might have:

- Books scattered everywhere
- Information about books written in different places
- No standard way to handle book operations (borrowing, returning, etc.)

```python
# The messy way (Procedural)
book1_title = "Harry Potter"
book1_author = "J.K. Rowling"
book1_isbn = "123456789"
book1_borrowed = False
```

```
def borrow_book(isbn, borrowed):
    if not borrowed:
        borrowed = True
        print(f"Book {isbn} has been borrowed")
    else:
        print("Book is already borrowed")
```

In an organised library (OOP):

- Books are neatly categorised (Classes)
- Each book has its proper place and information (Properties)
- Standard procedures for all book operations (Methods)

Python

```python
# The organized way (OOP)
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.borrowed = False

    def borrow(self):
        if not self.borrowed:
            self.borrowed = True
            print(f"Book '{self.title}' has been borrowed")
        else:
            print(f"Sorry, '{self.title}' is already borrowed")

    def return_book(self):
        if self.borrowed:
            self.borrowed = False
            print(f"Thank you for returning '{self.title}'")
        else:
            print("This book was not borrowed")

# Using the Book class
harry_potter = Book("Harry Potter", "J.K. Rowling", "123456789")
```

```
harry_potter.borrow()  # Book 'Harry Potter' has been borrowed
harry_potter.borrow()  # Sorry, 'Harry Potter' is already borrowed
harry_potter.return_book()  # Thank you for returning 'Harry Potter'
```

## Procedural Programming:

This programming paradigm follows a linear, top-down approach where the program is a sequence of instructions or procedures (functions). Each function performs a task and operates on data passed to it. Procedural programming is well-suited for smaller programs where the flow of execution is simple.

**Example**: A function calculate_area() might take the dimensions as input, perform the calculation, and return the result. Functions are isolated, and data is passed between them.

```python
def calculate_area(length, width):
    return length * width

area = calculate_area(5, 3)
print(area)  # Output: 15
```

## Object-Oriented Programming:

OOP focuses on modelling programs as collections of objects. These objects encapsulate data and functions (methods) that operate on the data. Instead of breaking the program into functions, OOP breaks it into classes and objects, promoting modularity, reusability, and scalability.

**Example**: A Shape class could have a method calculate_area(), and each shape (like a circle, square, or rectangle) can be represented as

an object of this class. Each object can call the method in a context-specific way.

```python
class Shape:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def calculate_area(self):
        return self.length * self.width

rectangle = Shape(5, 3)
print(rectangle.calculate_area())  # Output: 15
```

## Why Choose OOP over Procedural Programming?

OOP offers several advantages over procedural programming, especially for larger and more complex programs:

- **Modularity**: Objects and classes break down the program into manageable parts.
- **Reusability**: Once defined, classes and objects can be reused in different parts of the program or even in other programs.
- **Extensibility**: OOP allows you to easily extend programs by adding new classes or methods without affecting the existing code.

## Benefits of OOP

OOP provides several key benefits that make it a preferred choice for large, complex software systems:

- **Modularity:** OOP encourages breaking down a program into small, reusable components called classes. Each class is focused on a specific functionality, which makes the code easier to debug, test, and maintain. Modularity allows developers to isolate issues within a specific class, making troubleshooting much simpler.

  **Example:** In a library management system, different classes can represent `Books`, `Members`, and `Transactions`. Each class handles specific functions within the system, making the code organised and easier to manage.

- **Reusability:** OOP promotes code reusability through inheritance and polymorphism. Once a class is defined, it can be used to create multiple objects or extended to create subclasses without rewriting the existing logic.

  **Example:** A `Vehicle` class with attributes like `speed` and `fuel` can be extended to create subclasses like `Car`, `Bike`, and `Truck`, each inheriting these common attributes and methods.

- **Scalability and Flexibility**: OOP makes it easy to add new features or extend existing ones by creating new classes or modifying existing ones without disrupting other parts of the

system. This scalability is particularly beneficial in large, complex systems that evolve over time.

**Example:** In a customer relationship management (CRM) system, if there's a need to add a new `Lead` class, it can be seamlessly integrated without affecting other classes like `Customer` or `SalesRep`.

- **Data Security through Encapsulation:** Encapsulation in OOP restricts direct access to data by allowing access only through public methods (getters and setters). This helps in protecting sensitive data, enforcing data validation, and preventing unauthorised access.

  **Example:** In a `BankAccount` class, the balance attribute can be made private, and only accessible through specific methods. This prevents direct modification of the balance, which adds a layer of security.

- **Improved Collaboration with Clear Structure:** OOP provides a clear structure for dividing responsibilities among different classes, allowing teams to work on different parts of a project independently. This separation promotes collaboration and prevents conflicts in the code.

  **Example:** In a large e-commerce platform, developers can work on different classes such as `Product`, `Order`, and `User` simultaneously, which accelerates development while maintaining code integrity.

# Overview of OOP Principles

Object-Oriented Programming (OOP) is built on four core principles that guide how objects in programs are structured and how they interact with each other. These principles—**Encapsulation**, **Abstraction**, **Inheritance**, and **Polymorphism**—allow for the creation of robust, scalable, and maintainable code.

## Encapsulation

- **What:** Encapsulation means bundling the data (attributes) and the methods (functions) that operate on the data within one class, restricting direct access to some of the object's components.
- **Why:** It hides the internal workings of an object and only exposes what is necessary.
- **How:** Through access modifiers like private and public methods.

**Example:**

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    def deposit(self, amount):
        self.__balance += amount  # Public method
```

`__balance` is hidden and can only be modified by methods like `deposit()`.

## Abstraction

- **What:** Abstraction hides complex implementation details and only shows the necessary features of an object.

- **Why:** To reduce complexity and provide a simpler interface to interact with.
- **How:** Using abstract classes or interfaces.

**Example:**

```python
class Payment(ABC):
    @abstractmethod
    def process_payment(self):
        pass
```

The Payment class is abstract, meaning the details of process_payment() will be provided by subclasses like CreditCardPayment.

## Inheritance

- **What:** Inheritance allows one class to inherit properties and methods from another class.
- **Why:** Promotes code reuse and logical relationships between similar objects.
- **How:** By creating a child class that inherits from a parent class.

**Example:**

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound.")

class Dog(Animal):  # Dog inherits from Animal
    def speak(self):
        print(f"{self.name} barks.")
```

Dog inherits from Animal and can override the speak() method to provide its own behaviour.

## Polymorphism

- **What:** Polymorphism allows different classes to use the same interface while behaving differently.
- **Why:** Promotes flexibility and extensibility in code.
- **How:** Through method overriding and method overloading.

**Example:**

```python
class Bird:
    def move(self):
        print("Flying")

class Fish:
    def move(self):
        print("Swimming")
```

```
for animal in [Bird(), Fish()]:
    animal.move()  # Different behavior based on the object type
```

Both Bird and Fish implement move(), but behave differently when called.

# Python OOP Overview

Object-Oriented Programming (OOP) in Python is a powerful paradigm that enables developers to structure programs using objects. This approach leads to better-organised code, easier debugging, and scalability. Python, being an object-oriented language, provides a clean and flexible way to apply OOP principles through classes and objects.

## Introduction to Python OOP

### What is Python OOP?

Python is an object-oriented programming language, meaning it supports the concept of "objects"—entities that bundle data (attributes) and functionality (methods). OOP allows developers to model real-world entities as objects in code. In Python, you can define a class, which acts as a blueprint, and create instances of this class, called objects.

### Why Python for OOP?

Python's simple and easy-to-understand syntax makes it an excellent language for applying OOP principles. Its readability allows developers to focus on the logical structure of their code rather than

low-level implementation details. Python's dynamic nature also allows for flexible and concise object manipulation.

In Python, almost everything is treated as an object. Data types like integers, strings, lists, and even functions are instances of built-in classes.

```python
# Numbers are objects
x = 5
print(type(x))  # <class 'int'>

# Strings are objects
text = "Hello"
print(text.upper())  # Using a string method

# Lists are objects
my_list = [1, 2, 3]
my_list.append(4)  # Using a list method
```

## Basic Class Structure

Python classes allow us to create and manage complex objects with custom behaviour.

```python
class DataScientist:
    def __init__(self, name, specialization):
        self.name = name
        self.specialization = specialization

    def analyze_data(self, data):
        return f"{self.name} is analyzing {data} using
{self.specialization}"

# Using the DataScientist class
ds = DataScientist("Alice", "Machine Learning")
```

```
result = ds.analyze_data("customer data")
print(result)  # Output: Alice is analyzing customer data using
Machine Learning
```

# THANK YOU

AIMERZ.ai
Aim.Act.Achieve